Programación 1 — Unidad 9: Manejo de archivos

Actividad 4: Manejo de errores con archivos y formato JSON

Índice

9. Manejo de archivos

– 9.14. Qué puede salir mal al trabajar con archivos

– 9.15. Control de flujo con try / except

– 9.16. Excepciones específicas y genéricas

– 9.17. La filosofía de la «respuesta útil»

– 9.18. Otros formatos: JSON



9.14. Qué puede salir mal al trabajar con archivos

Hasta acá escribimos programas que funcionan **cuando todo sale bien**: el archivo existe, tiene el formato esperado y los datos se convierten sin problemas. En el mundo real eso casi nunca pasa. Un archivo puede no estar, puede estar corrupto, puede tener una columna de menos, o el usuario puede no tener permiso para leerlo.

Un programa **robusto** no es uno que nunca falla: es uno que **sabe qué hacer cuando falla**. En esta actividad vas a aprender a anticipar esos casos.

Estos son los errores más frecuentes al manipular archivos:

Excepción	Cuándo aparece	Ejemplo típico
`FileNotFoundError`	El archivo no existe en la ruta indicada	Abrir `datos.csv` en modo `"r"` sin que exista
`PermissionError`	No hay permisos para leer o escribir	Escribir en una carpeta protegida del sistema

Excepción	Cuándo aparece	Ejemplo típico
`ValueError`	Un dato no se puede convertir al tipo esperado	`int("edad")` al leer la cabecera por error
`IndexError`	Se pide una posición que no existe en la fila	`campos[2]` en una línea que solo tiene dos campos
`KeyError`	Se pide una clave que no está en el diccionario	`fila['precio']` cuando la columna se llama `costo`
`UnicodeDecodeError`	El encoding no coincide con el del archivo	Abrir un archivo con tildes sin `encoding="utf-8"`

Importante

Fijate que varios de estos errores **no dependen de tu código**, sino del estado del mundo: si el archivo está o no, si tenés permisos, si alguien editó el CSV a mano. Por eso no alcanza con «programar bien»: hay que **manejar el error explícitamente**.

9.15. Control de flujo con try / except

Abrir un archivo que no existe lanza un `FileNotFoundError` y **corta la ejecución** del programa. Conviene anticiparlo con un `try` / `except`, que es una estructura de control de flujo: le dice a Python «intentá esto, y si sale mal, hacé esto otro en lugar de romperte».

```
try:
    with open("datos.csv", "r", encoding="utf-8") as archivo:
        for linea in archivo:
            print(linea.strip())
except FileNotFoundError:
    print("No encontré el archivo datos.csv. Verificá el nombre y la ruta.")
```

Qué hace el código:

- El bloque `try:` contiene el código **que puede fallar**. Se ejecuta normalmente mientras no haya problemas.
- Si aparece un `FileNotFoundError`, Python **abandona el `try`** y salta directamente al `except` correspondiente.
- El programa **no se corta**: muestra el mensaje y sigue con lo que venga después.
- Si no hay ningún error, el bloque `except` sencillamente no se ejecuta.

El `try` / `except` admite dos bloques opcionales más, que ordenan mucho el código:

Bloque	Cuándo se ejecuta	Para qué sirve
`try`	Siempre; es el código vigilado	Contiene la operación riesgosa
`except`	Solo si se lanzó la excepción indicada	Maneja el error
`else`	Solo si **no** hubo ningún error	El código que depende de que el `try` haya salido bien
`finally`	**Siempre**, haya error o no	Limpieza y cierre de recursos

Buenas prácticas

Poné dentro del `try` **solo la línea o las líneas que pueden fallar**. Si metés cincuenta líneas adentro, cuando salte el error no vas a saber cuál de todas lo causó, y podés terminar atrapando errores que en realidad eran bugs tuyos.

9.16. Excepciones específicas y genéricas

Un mismo bloque `try` puede tener **varios** `except`, cada uno para un tipo de error distinto. Python los evalúa en orden y ejecuta el primero que coincida. Esto te permite dar una respuesta distinta según qué salió mal:

```
try:
    with open("notas.csv", "r", encoding="utf-8") as archivo:
        archivo.readline()          # salteamos la cabecera
        for linea in archivo:
            campos = linea.strip().split(",")
            print(campos[0], int(campos[1]))

except FileNotFoundError:
    print("No existe notas.csv. Generalo primero con la opción 1 del menú.")
except ValueError:
    print("Hay una nota que no es un número. Revisá el archivo.")
except IndexError:
    print("Hay una fila incompleta: se esperaban 2 columnas.")
```

Ahora bien: **no podés prever todos los errores posibles**. Siempre existe el imprevisto que no se te ocurrió. Para esos casos existe la **excepción genérica**, que atrapa cualquier cosa que no haya caído en los `except` anteriores:

```
try:
    with open("notas.csv", "r", encoding="utf-8") as archivo:
        procesar(archivo)

except FileNotFoundError:
    print("No existe notas.csv.")
except Exception as error:
    print(f"Ocurrió un problema inesperado: {error}")
```

Qué hace el código:

- `except Exception as error:` atrapa **cualquier** excepción que no haya sido capturada antes.
- `as error` guarda el objeto de la excepción en una variable, para poder mostrar de qué se trató.
- El orden importa: los `except` **específicos van primero** y el genérico **siempre al final**.
- Si pusieras `except Exception` arriba de todo, atraparía todo y los `except` específicos de abajo nunca se ejecutarían.

Importante

Usar `except Exception` como **única** forma de manejo es un antipatrón conocido. Esconde los errores en lugar de resolverlos: si tenés un typo en una variable, el genérico se lo traga y te muestra un mensaje amable mientras el bug sigue ahí. El genérico es la **red de seguridad**, no la estrategia principal.

Analogía

Pensalo como la guardia de un hospital. Los `except` específicos son los especialistas: traumatología, cardiología, pediatría. El `except Exception` es el médico clínico que atiende lo que no encaja en ninguna especialidad. Necesitás a los dos, pero si mandás **todos** los pacientes al clínico, nadie recibe el tratamiento correcto.

9.17. La filosofía de la «respuesta útil»

Atrapar el error es la mitad del trabajo. La otra mitad es **qué le decís al usuario**. Un mensaje como `Error` o `Algo salió mal` es casi tan inútil como que el programa se rompa: la persona no sabe qué hacer a continuación.

Un buen mensaje de error responde tres preguntas:

- **Qué pasó**, en palabras que el usuario entienda (no el nombre de la excepción en inglés).
- **Por qué pasó**, si lo sabés: el archivo no está, el dato no es un número, falta una columna.
- **Qué puede hacer** para resolverlo. Esta es la que más se olvida y la más valiosa.

Mensaje pobre	Mensaje útil
`Error`	`No encontré 'notas.csv' en esta carpeta. Verificá el nombre o generalo con la opción 1.`
`ValueError`	`La nota de la fila 4 no es un número válido. Revisá que no tenga letras ni espacios.`
`Falló la lectura`	`El archivo está en un formato distinto al esperado (se esperaban 3 columnas y hay 2).`

Buenas prácticas

No uses `print()` del objeto de excepción como único mensaje al usuario final. `[Errno 2] No such file or directory: 'notas.csv'` es información para vos, el programador. El usuario necesita: **qué pasó, por qué y qué hacer**. Podés mostrar las dos cosas, pero no confundas una con la otra.

Y un detalle que ya conocés: si usás `with`, el archivo se cierra solo aunque salte una excepción en el medio del bloque. Esa es otra razón para que sea tu forma por defecto de trabajar.

Buenas prácticas

Combiná lo aprendido: envolvé las operaciones con archivos en un `try` / `except` para manejar el `FileNotFoundError` cuando el archivo no exista. Así tu programa avisa con claridad en lugar de romperse.

9.18. Otros formatos: JSON

El CSV es perfecto para tablas, pero se queda corto cuando los datos tienen **estructura anidada**: un diccionario que adentro tiene una lista, o una lista de diccionarios con distintos campos. Para eso el formato estándar es **JSON**, que se maneja con el módulo `json` de la biblioteca estándar.

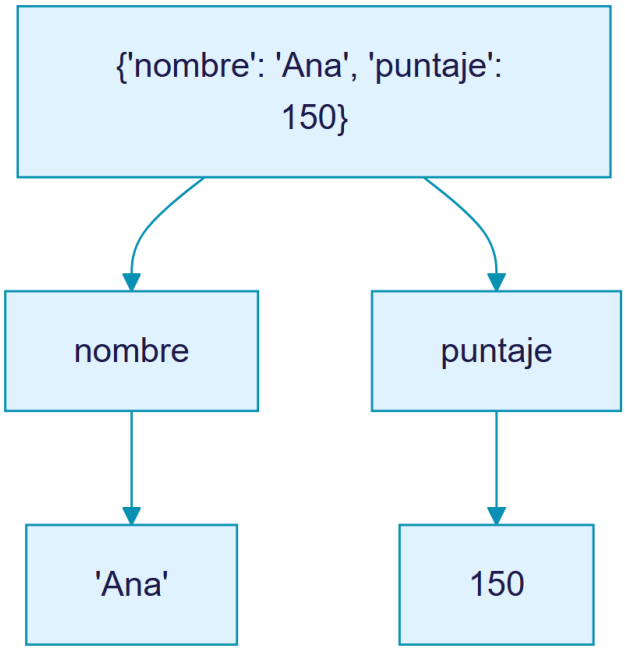


Diagrama 9.18: JSON conserva la estructura completa de un diccionario, con sus claves y sus valores.

```
import json

datos = {"nombre": "Ana", "notas": [7, 8, 10]}
with open("datos.json", "w", encoding="utf-8") as f:
    json.dump(datos, f)

with open("datos.json", "r", encoding="utf-8") as f:
    recuperado = json.load(f)

print(recuperado["notas"])   # [7, 8, 10]
```

Qué hace el código:

- `datos` es un diccionario que adentro tiene una lista: una estructura que JSON guarda sin problemas.
- `json.dump(datos, f)` convierte ese diccionario en texto JSON y lo escribe en el archivo.
- `json.load(f)` hace lo inverso: lee el archivo y reconstruye el diccionario original en `recuperado`.
- Por eso `recuperado["notas"]` devuelve la lista `[7, 8, 10]` tal como la habíamos guardado.

Fijate la diferencia con el CSV: al leer un CSV **todo vuelve como string** y hay que convertir a mano. JSON, en cambio, **conserva los tipos**: los números vuelven como números y las listas como listas.

	CSV	JSON
Forma de los datos	Tabla plana (filas y columnas)	Estructuras anidadas (dict y listas)
Tipos de datos	Todo vuelve como texto	Conserva números, booleanos y listas
Se abre con Excel	Sí	No
Ideal para…	Listados, registros, exportaciones	Configuraciones, respuestas de APIs, datos anidados

Nota

JSON también tiene sus propios errores. Si el archivo existe pero su contenido está mal formado, `json.load()` lanza un `json.JSONDecodeError`. Es un buen candidato para un `except` específico, junto al `FileNotFoundError`.

Encontrá el error

Este programa falla apenas intenta escribir. Fijate en qué modo se abrió el archivo antes de seguir.

```
archivo = open("resultado.txt", "r")
archivo.write("Nueva linea de resultados")
archivo.close()
```

Qué estaba mal

El archivo se abrió en modo "r" (solo lectura), así que no se puede escribir en él: Python lanza un error de operación no soportada. Para escribir hace falta abrirlo en modo "w" o "a", y lo recomendable es usar with para que se cierre solo.

Practicá la lectura

Explicá con tus palabras qué hace json.load y qué tendría que tener el archivo config.json para que este programa funcione. ¿Qué dos excepciones distintas podría lanzar?

```
import json

with open("config.json", "r", encoding="utf-8") as f:
    datos = json.load(f)

print(datos["volumen"])
```

Para reflexionar y autoevaluarte

— ¿Qué ventaja concreta le ves a usar `with` en vez de abrir y cerrar el archivo a mano?

— ¿Por qué conviene poner los `except` específicos antes del genérico?

— ¿Qué formato (texto, CSV o JSON) elegirías para guardar los datos de un programa tuyo, y por qué?

Antes de seguir, verificá que podés:

— Abrir, leer y escribir archivos usando `with`.

— Elegir el modo de apertura correcto (`r`, `w`, `a`) según lo que se necesita hacer.

— Leer y escribir archivos CSV con el módulo `csv`.

— Envolver una operación riesgosa en un `try` / `except` y manejar la excepción específica.

— Escribir un mensaje de error que le sirva al usuario para resolver el problema.

— Guardar y recuperar estructuras de datos completas con JSON.

Con esto cerramos la Unidad 9. Ya podés hacer que la información de tus programas **sobreviva**, elegir el formato adecuado para cada caso y que nada de eso se rompa cuando el mundo real no coopera.